

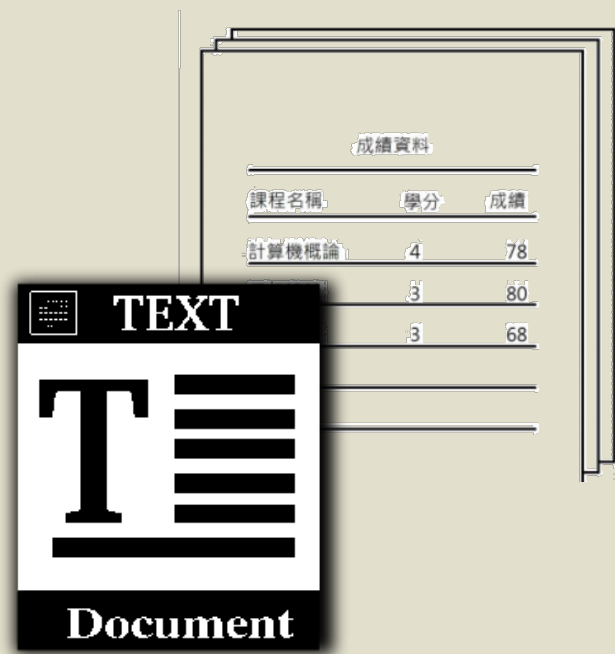
PYTHON機器學習入門

DATA ANALYSIS PACKAGE

授課教師：江尚瑀

數據分析概念

- 數據(Data) :
 - 原始資料，一個未經過任何處理的文字與數字(數字性的特徵或資訊)
 - 事實的記錄，沒有特定的意義



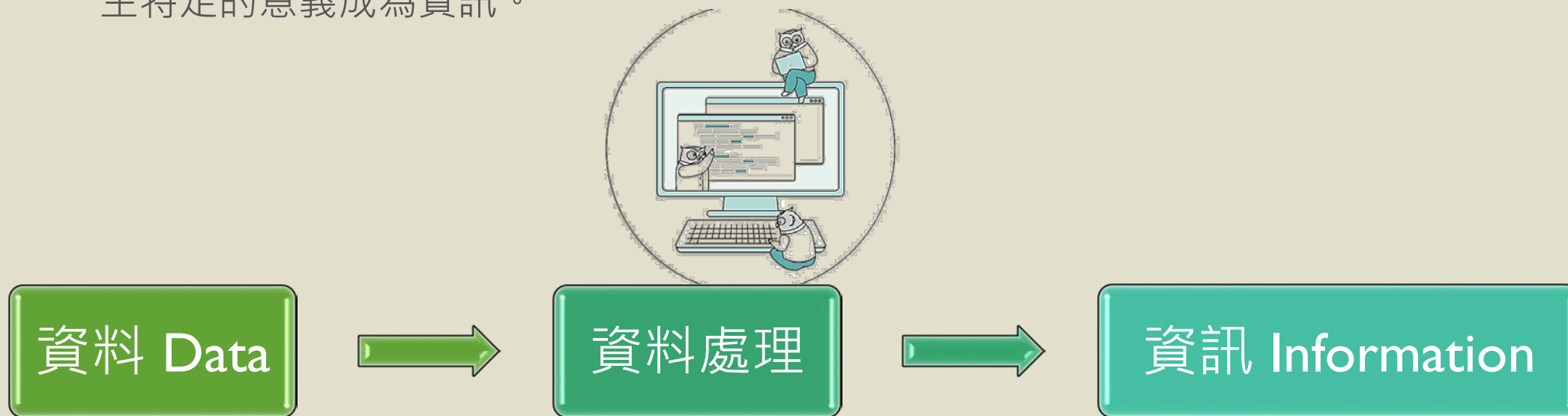
數據分析概念

- 數據分析

- 即資料處理(**data analysis**)，目的為如何獲取有效的資訊，並藉以做出正確的決策，來達到預期的成果。

- 資訊

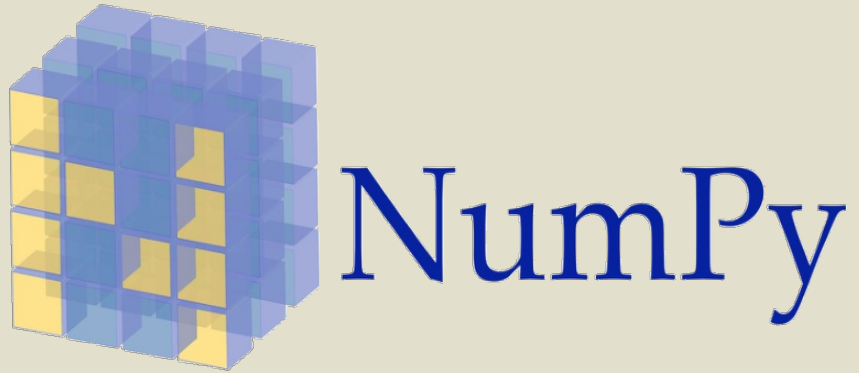
- 有意義的資料，資料依據某些規則被分類、歸納後，再加以分析、解釋，最後會產生特定的意義成為資訊。



PYTHON數據分析工具

- **NumPy**：Python做多維陣列（矩陣）運算時的必備套件，比起Python內建的list，NumPy的array有極快的運算速度優勢。
- **Scipy**：是基於numpy的科學計算核心包，它能與NumPy陣列一起工作，並提供了許多使用者友好和高效的數字實踐，如實現插值，積分，優化，影象處理等等。
- **Pandas**：有了Pandas可以讓Python很容易做到幾乎所有Excel的功能了，像是樞紐分析表、小記、欄位加總、篩選
- **Matplotlib**：基本的視覺化工具，可以畫長條圖、折線圖等等。
- **Seaborn**：另一個知名的視覺化工具，有些人認為畫起來比matplotlib好看。
- **SciKit-Learn**：Python 關於機器學習的model基本上都在這個套件，像是SVM, Random Forest...

OUTLINE

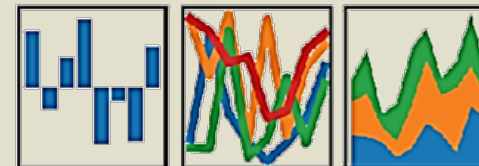


- **NumPy** is the fundamental package for scientific computing with Python.
- It contains: a powerful **N-dimensional array object, sophisticated functions, tools for integrating C/C++ and Fortran code, useful linear algebra, Fourier transform, and random number capabilities.**



- **pandas** is an open source, BSD-licensed library providing high-performance, **easy-to-use data structures and data analysis tools** for the Python programming language.

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$



	BandName	WavelengthMax	WavelengthMin
0	CoastalAerosol	450	430
1	Blue	510	450
2	Green	590	530
3	Red	670	640
4	NearInfrared	880	850
5	ShortWaveInfrared_1	1650	1570
6	ShortWaveInfrared_2	2290	2110
7	Cirrus	1380	1360

A decorative wavy line in a light green color runs vertically along the left side of the slide, separating a light beige background from the dark teal background.

PYTHON PACKAGE - NUMPY

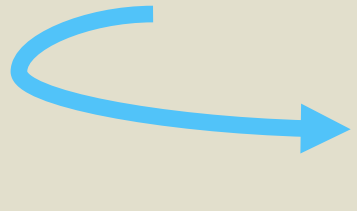
PART I

認識 NUMPY

- What is Numpy?

NumPy是python的一種開放原始碼的數值計算擴充，用來儲存和處理多維陣列，核心的資料結構是ndarray

- What is ndarray



ndarray, a fast and space-efficient multidimensional array providing vectorized arithmetic operations and sophisticated broadcasting capabilities

ndarray是一個快速的且可以節省空間的多維度陣列
提供向量運算以及複雜的廣播功能

- ndarray建置的方法

1.引入numpy的套件、2.建立資料Data、3.資料基本運算

認識 NUMPY

- Numpy 陣列建置

同質且多重維度的 ndarray (N-dimensional array) 。

一般我們稱一維陣列為 vector 而二維陣列為 matrix 。

1 D ARRAY:

C	O	D	I	N	G	E	E	K
0	1	2	3	4	5	6	7	8

← single row of elements

2 D ARRAY:

		col 0	col 1	col 2	
	i \ j	0	1	2	
row 0	0	A	A	A	} array elements
row 1	1	B	B	B	
row 2	2	C	C	C	

↑ rows

← column

一開始我們會引入 numpy 模組，
透過傳入 list 到 numpy.array() 創建陣列。

```
In [1]: import numpy as np
```

在匯入 NumPy 套件後
可以使用 array() 方法建立 NumPy 陣列

```
In [2]: a = np.array([1, 2, 3, 4, 5])
print(type(a))
print(a[0], a[1], a[2], a[3], a[4])

<class 'numpy.ndarray'>
1 2 3 4 5
```

認識 NUMPY

- Numpy 陣列

- ndarray 的關鍵屬性是維度 (ndim) 、形狀 (shape) 和數值類型 (dtype) 。

讀取方式，橫(左右)的為列 row，直(上下)的為欄 column(col)

陣列	說明
一維陣列	向量(Vector)
二維陣列	矩陣(matrix)
多維陣列	使用多個矩陣(matrix)各軸向(axis)組合起來的形狀(shape)

一維陣列	1D array	向量(Vector)
1	2	3

二維陣列	2D array	矩陣(matrix)
1	2	3
4	5	6

*1D陣列_____

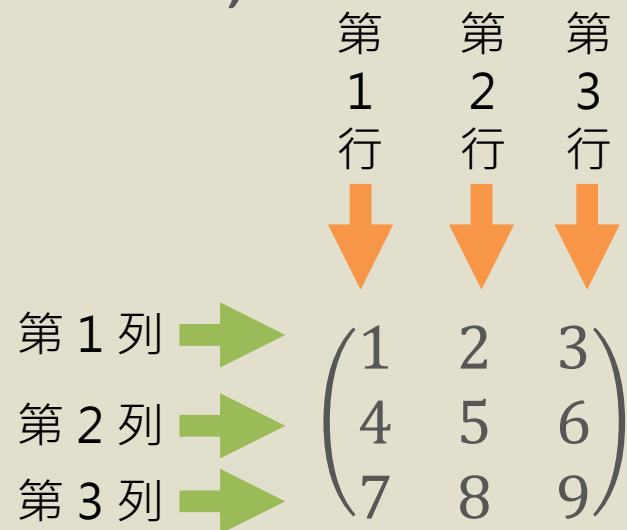
*2D陣列_____

*nD多維陣列_____

矩陣表達方式

- 矩陣的列與行方式

Matrix = row x col (column)



$m \times n$ 矩陣， $m \rightarrow \text{row}$ $n \rightarrow \text{col}$

$$2 \times 3 \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \quad 3 \times 2 \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

ARRAY VS. LIST

Differences between an array and a list

array

can hold only homogeneous data

not a native data type (need an import)

arithmetic operations are element-wise

uses memory efficiently

operation on arrays are typically faster

a loop has to be formed to print or access the components of a array

adding elements to a numpy array creates a new array and destroys the old one.

list

can hold heterogeneous data

native data type (no need to import)

arithmetic operations are not element-wise

not efficient in memory usage

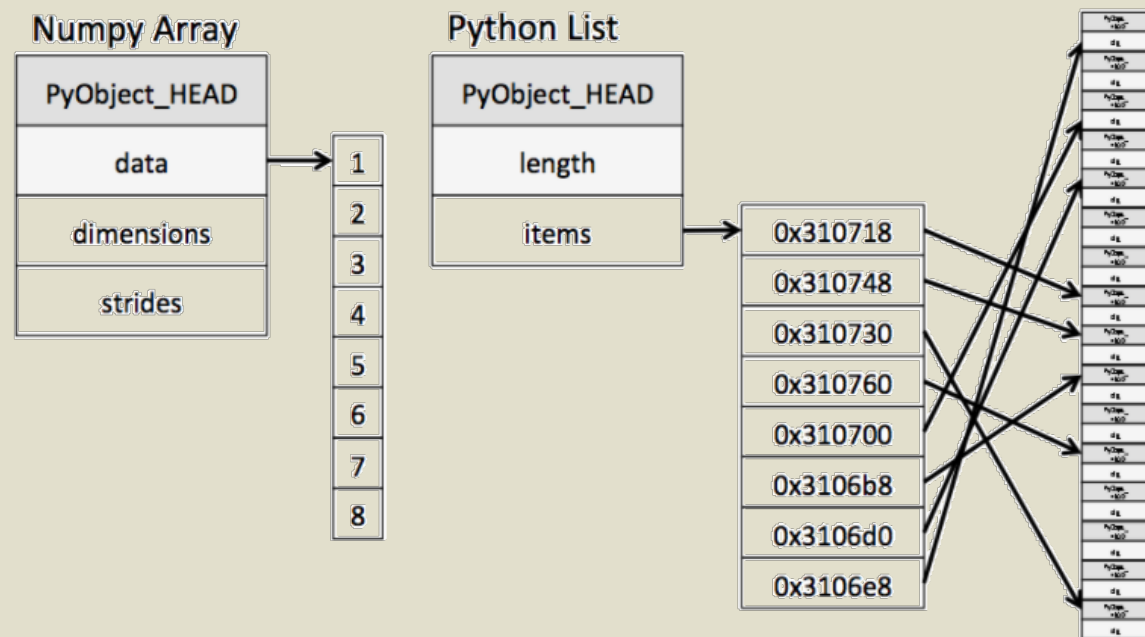
operation on lists are typically slower

the entire list can be printed without any explicit looping

Elements can be added to a list without creating a whole new list.

ARRAY VS. LIST

- **array**利用底層C語言會用的記憶體操作方法，效益是比較快也比較省空間
 - 一開始就定義好陣列內部的型態定義，所以不需要每次都到底層的 **vtable**去做比對
 - 要記憶體方式不同：不同於 **list**，**array**在要記憶體是直接要一大塊，並且把資料直接儲存在這個空間內，這樣便少了一個 **list** 和 **pointer** 的儲存空間，也就比較省空間，且要到資料的速度也比較快了。



建立陣列

- 使用清單與元組建立一維陣列
就可以使用索引來更改指定的陣列元素值，如下所示：

```
In [3]: a[0] = 5  
        print(a)  
        a[3] = 0  
        print(a)  
  
[5 2 3 4 5]  
[5 2 3 0 5]
```

- 使用巢狀清單建立二維陣列
NumPy二維陣列是使用巢狀清單來建立，如下所示：

```
In [4]: b = np.array([[1,2,3],[4,5,6]])  
        print(b[0, 0], b[0, 1], b[0, 2])  
        print(b[1, 0], b[1, 1], b[1, 2])  
  
1 2 3  
4 5 6
```

建立陣列

- 如何指定巢狀清單二維陣列**Index**

二維陣列的陣列索引值有**2**個：**[左索引值, 右索引值]**，需要使用**2**個索引值來存取二維陣列的指定元素值，可以更改左上角和右下角的**2**個元素值。

```
In [5]: b[0, 0] = 6  
        b[1, 2] = 1  
        print(b)
```

```
[[6 2 3]  
 [4 5 1]]
```

建立陣列

- 如何建立指定元素型態的陣列

在建立**NumPy**陣列時，可以指定陣列元素是哪一種資料型態的陣列，如下所示：

```
In [6]: c = np.array([1, 2, 3, 4, 5], int)
        print(c)
        d = np.array((1, 2, 3, 4, 5), dtype=float)
        print(d)
```

```
[1 2 3 4 5]
[1. 2. 3. 4. 5.]
```

更多NUMPY陣列的建立方法

- NumPy提供多種方法來建立預設內容的陣列，NumPy的`arange()`方法類似Python的`range()`函數，可以產生一序列數字的陣列，如下所示：

```
In [2]: a = np.arange(5)
        print(a)
        b = np.arange(1, 6, 2)
        print(b)

[0 1 2 3 4]
[1 3 5]
```

- `zeros()`方法可以產生指定個數元素值都是0的一維和二維陣列，`ones()`方法可以產生指定個數元素值都是1的一維和二維陣列，如下所示：

```
In [3]: c = np.zeros(2)
        print(c)
        d = np.zeros((2,2))
        print(d)

[0. 0.]
[[0. 0.]
 [0. 0.]]
```

```
In [4]: e = np.ones(2)
        print(e)
        f = np.ones((2,2))
        print(f)

[1. 1.]
[[1. 1.]
 [1. 1.]]
```

陣列維度轉換

- **NumPy**可以使用**reshape()**方法，將一維陣列轉換成二維陣列。

```
In [2]: a = np.arange(16)
        print(a)
        b = a.reshape((4, 4))
        print(b)
```

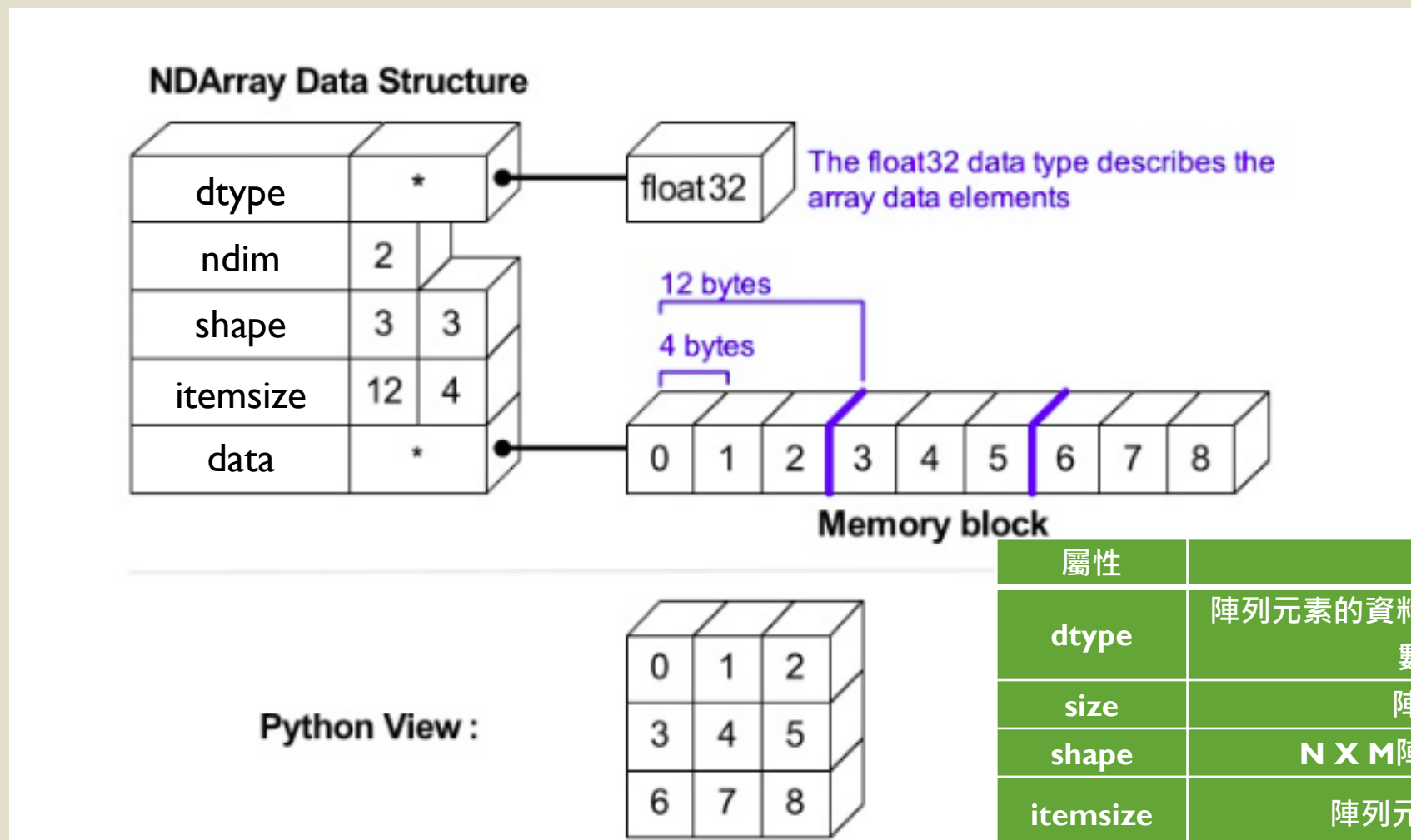
```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
```

陣列屬性

- **NumPy**陣列是物件，提供相關屬性顯示陣列資訊。
相關屬性的說明如下表所示：

屬性	說明
dtype	陣列元素的資料型態，整數int32/64或浮點數float32/64等
size	陣列的元素總數
shape	N X M 陣列的形狀（ Shape ）
itemsize	陣列元素佔用的位元組數
ndim	幾維陣列，一維是1；二維是2
nbytes	整個陣列佔用的位元組數

陣列屬性



屬性	說明
dtype	陣列元素的資料型態，整數int32/64或浮點數float32/64等
size	陣列的元素總數
shape	N X M 陣列的形狀（Shape）
itemsize	陣列元素佔用的位元組數
ndim	幾維陣列，一維是2；二維是2
nbytes	整個陣列佔用的位元組數

走訪陣列的元素

- 走訪一維陣列的元素 - **for**迴圈

```
In [2]: a = np.array([1, 2, 3, 4, 5])
        for ele in a:
            print(ele, end=" ")
```

1 2 3 4 5

- 走訪二維陣列的元素 - **for**巢狀迴圈

```
In [3]: b = np.array([[1, 2], [3, 4], [5, 6]])
        for ele in b:
            print(ele, end=" ")
        print()
        for ele in b:
            for item in ele:
                print(item, end=" ")
```

[1 2] [3 4] [5 6]
1 2 3 4 5 6

使用亂數函數產生陣列元素值

- **NumPy**的**random**子模組提供多種函數來產生亂數，可以產生一整個陣列元素值的亂數值，相關方法的說明如下表所示：

方法	說明
seed(int)	指定亂數的種子數，這是整數值，同一個種子數會產生相同的亂數序列
random()	產生0.0~1.0之間的亂數
randint(min,max,size)	產生min~max之間的整數亂數，不含max，如果有第3個參數可以產生整數亂數值的陣列
rand(row,col)	產生亂數值的陣列，第1個參數是一維陣列的尺寸，第2個參數是二維陣列的列與欄
randn(row,col)	類似rand()方法，可以產生標準常態分配的樣本資料

向量的基礎觀念

- 位置

單純一個數字，**沒有**大小或方向，可以用實數表達。

- 認識向量

一個同時具有**大小**與**方向**的物件。

二維空間(平面座標) -> 包含**2**個元素分別為**X**軸(橫座標)**y**軸(縱座標)

三維空間 -> 向量即包含**X**軸(橫座標)**y**軸(縱座標)**Z**軸座標

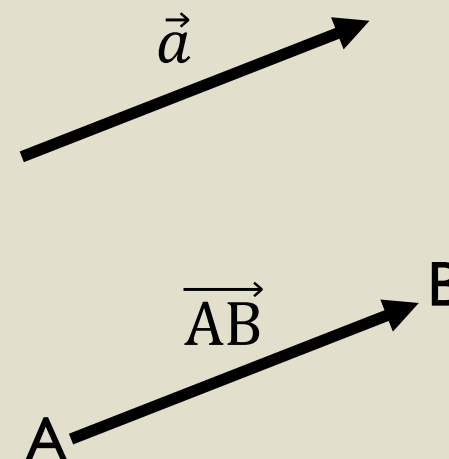
.....

N維空間 -> 向量的元素個數是**n**個，**向量分量**也是**n**個

- 向量表示

含箭頭的線條表表示，**線條長度**代表向量**大小**，**箭頭**代表向量**方向**
等向量大小方向均相同，位置可以相同也可以不相同

文字表示 $\vec{a}$ ，含起點與終點表示成 $\overrightarrow{AB}$ (有方向性)



向量運算

- 向量與純量的四則運算

- 向量與純量 (**Scalar**) 可以進行加、減、乘和除的四則運算，純量是一個數值。

```
In [2]: a = np.array([1, 2, 3])  
print(a)  
s = 5  
print(s)
```

```
[1 2 3]  
5
```

```
In [3]: c = a + s  
print(c)  
c = a - s  
print(c)  
c = a * s  
print(c)  
c = a / s  
print(c)
```

```
[6 7 8]  
[-4 -3 -2]  
[ 5 10 15]  
[0.2 0.4 0.6]
```

向量運算的規則

- 相同維度的向量才可以相加

$a = (a_1, a_2, a_3, \dots, a_n)$

$b = (b_1, b_2, b_3, \dots, b_n)$

$c = (c_1, c_2, c_3, \dots, c_n)$

- $a + b = b + a$ 交換律

-> 改變 順序 不改變 結果

- $(a + b) + c = a + (b + c)$

結合律 運算子的位置

-> 沒有變運算順序不會改變結果

- $a + z = a$

-> z 是零向量， z 亦可標記為 0

- $a + b = 0$

-> 反向量 一個大小相等但是方向相反的向量

```
In [5]: s = np.array([4, 5, 6])
print(s)
c = a + s
print(c)
c = a - s
print(c)
c = a * s
print(c)
c = a / s
print(c)
```

```
[4 5 6]
[5 7 9]
[-3 -3 -3]
[ 4 10 18]
[0.25 0.4 0.5 ]
```

```
>>> import numpy as np
>>> a = np.array([3,2])
>>> -a
array([-3, -2])
```

向量運算的規則

- 向量與純量相乘

$$\mathbf{a} = (a_1, a_2, a_3, \dots, a_n)$$

純量 $c \times y$

$$c * \mathbf{a} = (ca_1, ca_2, \dots, ca_n)$$

```
array([-3, -2])  
>>> import numpy as np  
>>> a = np.array([3, 2])  
>>> 3 * a  
array([9, 6])
```

- 點積運算 (**Dot Product**)

是兩個向量對應元素的乘積和，向量 $\mathbf{a}$ 和 $\mathbf{s}$ 的點積運算結果是一個純量

$$\mathbf{a} = [a_1, a_2, a_3]$$

$$\mathbf{s} = [s_1, s_2, s_3]$$

$$c = \mathbf{a} \bullet \mathbf{s} = a_1 \times s_1 + a_2 \times s_2 + a_3 \times s_3$$

```
In [6]: c = a.dot(s)  
print(c)
```

32

- 分配律

$$(\mathbf{x} + \mathbf{y}) * \mathbf{a} = \mathbf{x}\mathbf{a} + \mathbf{y}\mathbf{a} ; \mathbf{x}(\mathbf{a} + \mathbf{b}) = \mathbf{x}\mathbf{a} + \mathbf{x}\mathbf{b} ; (\mathbf{x}\mathbf{y}) \mathbf{a} = \mathbf{x}(\mathbf{y}\mathbf{a})$$

- 其他

$$\mathbf{a} * 1 = \mathbf{a} ; \mathbf{a} * -1 = -\mathbf{a} ; \mathbf{a} * 0 = 0 ; \mathbf{a} - \mathbf{b} = \mathbf{a} + (-\mathbf{b})$$

向量的長度

- 使用numpy的linalg模組的norm方法處理
`np.linalg.norm(x, ord=None, axis=None, keepdims=False)`

参数	说明	计算方法
默认	二范数:	$\sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$
ord=2	二范数:	同上
ord=1	一范数:	$ x_1 + x_2 + \dots + x_n $
ord=np.inf	无穷范数:	$MAX x_i $

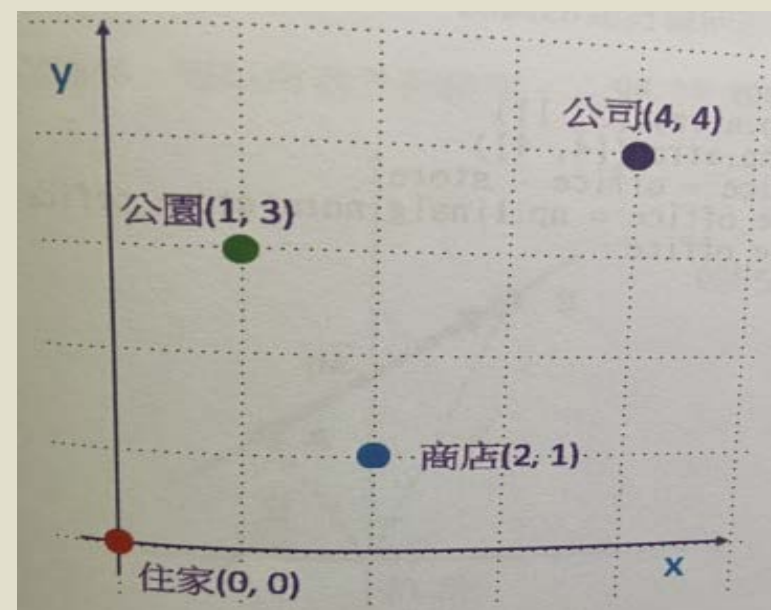
- 從住家到公園的距離

$$\sqrt{1^2 + 3^2} = \sqrt{10}$$

```
>>> import numpy as np
>>> park = np.array([1,3])
>>> norm_park = np.linalg.norm(park)
>>> norm_park
3.1622776601683795
```

- 從商店到公司的距離

$$\sqrt{(4-2)^2 + (4-1)^2} = \sqrt{13}$$



切割一維陣列的元素

- NumPy陣列一樣可以使用**list**的切割運算子

```
In [2]: a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
        print(a)
        b, c, d = a[1:3], a[:4], a[3:]
        print(b, c, d)
```

```
[1 2 3 4 5 6 7 8 9]
[2 3] [1 2 3 4] [4 5 6 7 8 9]
```

- 在切割運算子共有**3**個參數，其語法 **array[start:end:step]**
上述「:」冒號分隔的值是**start**至**end**的範圍，**step**是增量，如下所示：

```
In [3]: b, c, d = a[2:9:3], a[::2], a[::-1]
        print(b, c, d)
```

```
[3 6 9] [1 3 5 7 9] [9 8 7 6 5 4 3 2 1]
```

矩陣運算

- 矩陣與純量的四則運算

- 矩陣與純量 (**Scalar**) 可以進行加、減、乘和除的四則運算，純量是一個數值。
- 矩陣與純量適用運算子 $+$ 、 $-$ 、 $*$ 和 $/$ 的四則運算

```
In [2]: a = np.array([[1,2,3],[4,5,6]])  
print(a)  
s = 5  
print(s)
```

```
[[1 2 3]  
 [4 5 6]]  
5
```

```
In [3]: c = a + s  
print(c)  
c = a - s  
print(c)  
c = a * s  
print(c)  
c = a / s  
print(c)
```

```
[[ 6  7  8]  
 [ 9 10 11]]  
[[-4 -3 -2]  
 [-1  0  1]]  
[[ 5 10 15]  
 [20 25 30]]  
[[0.2 0.4 0.6]  
 [0.8 1.  1.2]]
```

矩陣運算

- 如果有相同形狀的**2**個矩陣，對應的矩陣元素也可以執行加、減、乘和除的四則運算來產生相同形狀的矩陣

```
In [4]: a = np.array([[1,2],[3,4]])  
print(a)  
s = np.array([[5,6],[7,8]])  
print(s)
```

```
[[1 2]  
 [3 4]]  
[[5 6]  
 [7 8]]
```

In [5]:

```
c = a + s  
print(c)  
c = a - s  
print(c)  
c = a * s  
print(c)  
c = a / s  
print(c)
```

```
[[ 6  8]  
 [10 12]]  
[[-4 -4]  
 [-4 -4]]  
[[ 5 12]  
 [21 32]]  
[[0.2      0.33333333]  
 [0.42857143 0.5      ]]
```

矩陣運算

- 矩陣的點積運算

- 點積運算 (**Dot Product**) 是兩個矩陣對應元素的列和行的乘積和。
- 矩陣 **a** 和 **s** 點積運算的結果是另一個矩陣

$$a = \begin{bmatrix} a1 & a2 \\ a3 & a4 \end{bmatrix}$$
$$s = \begin{bmatrix} s1 & s2 \\ s3 & s4 \end{bmatrix}$$

$$c = a \bullet s = \begin{bmatrix} a1 * s1 + a2 * s3 & a1 * s2 + a2 * s4 \\ a3 * s1 + a4 * s3 & a3 * s2 + a4 * s4 \end{bmatrix}$$

```
In [6]: c = a.dot(s)
print(c)

[[19 22]
 [43 50]]
```

切割二維陣列的元素

- **NumPy**二維陣列一樣可以使用切割運算子，從原始陣列切割出所需的子陣列，其語法如下所示：

array[start:end:step, start1:end1:step1]

- 上述**2**個索引，分別都是開始、結束（不包含結束本身）和增量。

Q: 程式碼建立???陣列???~???後，使用**reshape()**方法轉換成二維陣列？ X？

```
In [2]: a = np.arange(11,36)
a = a.reshape(5,5)
print(a)
```

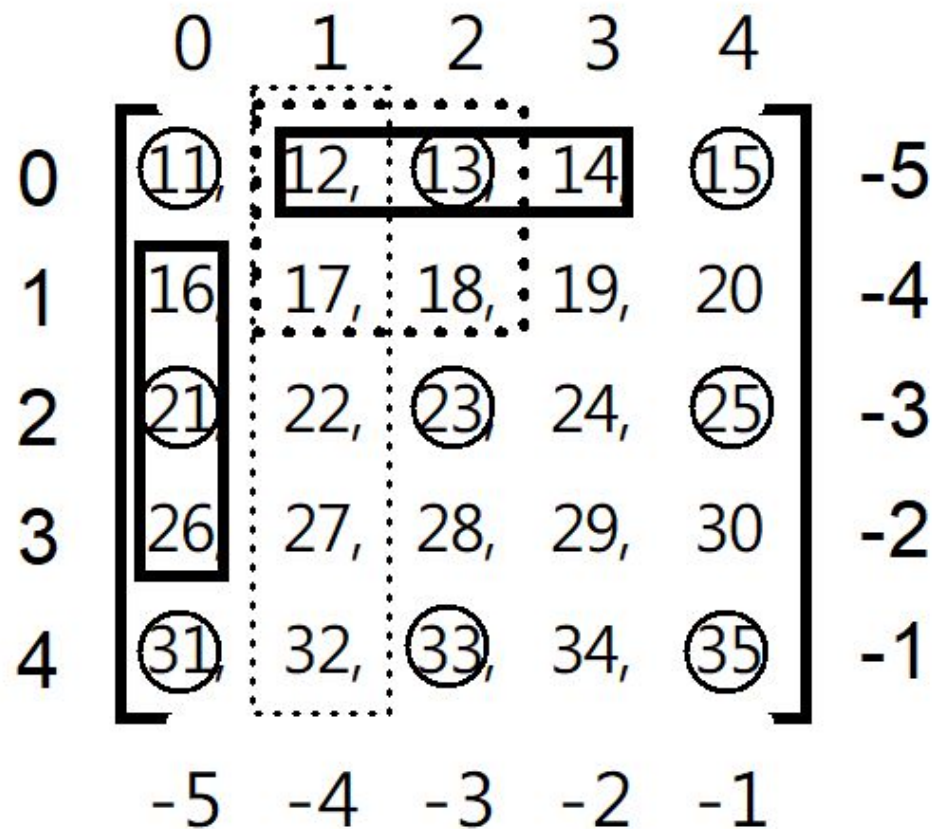
```
[[11 12 13 14 15]
 [16 17 18 19 20]
 [21 22 23 24 25]
 [26 27 28 29 30]
 [31 32 33 34 35]]
```

切割二維陣列的元素

Q: 程式碼建立一維陣列**11~36**後，使用**reshape()**方法轉換成二維陣列**5 X 5**

```
In [2]: a = np.arange(11,36)
a = a.reshape(5,5)
print(a)
```

```
[[11 12 13 14 15]
 [16 17 18 19 20]
 [21 22 23 24 25]
 [26 27 28 29 30]
 [31 32 33 34 35]]
```



切割二維陣列的元素 – 範例

- 第一個切割範例：二維陣列的列索引是**0,0,0**；欄索引是**1,2,3**，可以取出**[0,1]**、**[0,2]**和**[0,3]**的**3**個元素，如下所示：

```
In [3]: b = a[0, 1:4]
        print(b)

[12 13 14]
```

	0	1	2	3	4	
0	⑪	12, 13, 14	⑫	⑬	⑭	-5
1	16	17, 18, 19	20	21	22	-4
2	23	24, 25, 26	27	28	29	-3
3	30	31, 32, 33	34	35	36	-2
4	37	38, 39, 40	41	42	43	-1
	-5	-4	-3	-2	-1	

- 第二個切割範例：二維陣列的列索引是**1,2,3**；欄索引是**0,0,0**，可以取出**[1,0]**、**[2,0]**和**[3,0]**的**3**個元素，如下所示：

```
In [4]: b = a[1:4, 0]
        print(b)

[16 21 26]
```

	0	1	2	3	4	
0	⑪	12, 13, 14	⑫	⑬	⑭	-5
1	16	17, 18, 19	20	21	22	-4
2	23	24, 25, 26	27	28	29	-3
3	30	31, 32, 33	34	35	36	-2
4	37	38, 39, 40	41	42	43	-1
	-5	-4	-3	-2	-1	

切割二維陣列的元素 – 範例

- 第三個切割範例：二維陣列的列索引是**0,1**；欄索引是**1,2**，可以取出**[0,1]**、**[0,2]**、**[1,1]**和**[1,2]**的4個元素，如下所示：

```
In [5]: b = a[:2, 1:3]
print(b)

[[12 13]
 [17 18]]
```

	0	1	2	3	4	
0	⑪	12, ⑬	14	⑮	-5	
1	16	17, 18,	19	20	-4	
2	21	22, ⑲	24,	25	-3	
3	26	27,	28,	29,	30	-2
4	⑳	32,	33,	34,	35	-1
	-5	-4	-3	-2	-1	

- 第四個切割範例：二維陣列的列索引是**0,1,2,3,4**；欄索引是**1,1,1,1**，可以取出**[0,1]**、**[1,1]**、**[2,1]**、**[3,1]**和**[4,1]**的5個元素，如下所示：

```
In [6]: b = a[:,1]
print(b)

[12 17 22 27 32]
```

	0	1	2	3	4	
0	⑪	12, ⑬	14	⑮	-5	
1	16	17, 18,	19	20	-4	
2	21	22, ⑲	24,	25	-3	
3	26	27,	28,	29,	30	-2
4	31	32,	33,	34,	35	-1
	-5	-4	-3	-2	-1	

切割二維陣列的元素 – 範例

- 第五個切割範例：二維陣列的列索引是0,2,4；欄索引是0,2,4，可以取出[0,0]、[0,2]、[0,4]、[2,0]、[2,2]、[2,4]、[4,0]、[4,2]和[4,4]的9個元素，如下所示：

```
In [7]: b = a[:, ::2, ::2]  
print(b)
```

```
[[11 13 15]  
 [21 23 25]  
 [31 33 35]]
```

	0	1	2	3	4	
0	⑪	12, 13, 14	⑬	15	-5	
1	16	17, 18, 19	20	-4		
2	21	22, 23	24, 25	-3		
3	26	27, 28, 29	30	-2		
4	31	32, 33	34, 35	-1		
	-5	-4	-3	-2	-1	

使用進階索引(INDEX)取出元素

- 使用整數值清單的索引

- **NumPy**陣列不只可以使用整數值的索引來取出指定值，還可以給一個索引清單，取出選擇的元素來建立一個新陣列

```
In [2]: a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
        print(a)
        b = a[[1, 3, 5, 7]]
        print(b)

[1 2 3 4 5 6 7 8 9]
[2 4 6 8]
```

- **NumPy**陣列還可以使用相同大小的布林值陣列，如果元素值是**True**，表示選擇對應元素；反之**False**，就是不選擇，

```
In [7]: b = np.array([1, 2, 3, 4, 5, 6])
        mask = (b % 2 == 0)
        print(mask)

[False  True False  True False  True]
```

小練習: **ARRAY** 的操作索引 & 局部資料(切片)

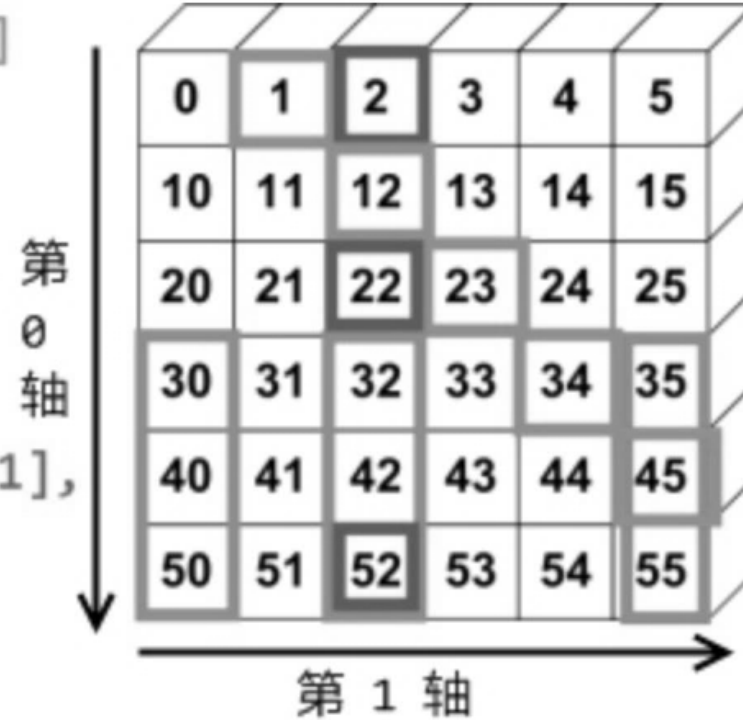
方法	說明
<code>.[index]</code>	索引 · index 從 0 開始 · -1 表示倒數第一個
<code>.[start, end, dist]</code>	切片 · 從 start 到 end · 間距為 dist

```
>>> a[0,3:5]
array([3,4])
>>> a[4:,4:]
array([[44,45],[54,55]])
>>> a[:,2]
array([2,12,22,32,42,52])
>>> a[2::2,::2]
array([[20,22,24],
       [40,42,44]])
```

0	1	2	3	4	5
10	11	12	13	14	15
20	21	22	23	24	25
30	31	32	33	34	35
40	41	42	43	44	45
50	51	52	53	54	55

小練習: **ARRAY**的操作索引 & 局部資料(切片)

```
>>> a[(0,1,2,3,4),(1,2,3,4,5)]  
array([1,12,23,34,45])  
>>> a[3:,[0,2,5]]  
array([[30,32,35],  
       [40,42,45],  
       [50,52,55]])  
>>> mask=np.array([1,0,1,0,0,1],  
                   dtype=np.bool)  
>>> a[mask,2]  
array([2,22,52])
```



陣列形狀與內容操作

- 陣列平坦化的`ravel()`方法可以將二維陣列平坦化成一維陣列
- NumPy**還可以使用`.T`屬性或`transpose()`方法來交換陣列維度

```
In [2]: a = np.array([[1,2,3],[4,5,6]])  
print(a)  
b = a.ravel()  
print(b)  
b = np.ravel(a)  
print(b)
```

```
[[1 2 3]  
 [4 5 6]]  
[1 2 3 4 5 6]  
[1 2 3 4 5 6]
```

```
In [3]: b = np.array([[1,2],[3,4],[5,6]])  
print(b)  
c = b.T  
print(c)  
c = b.transpose()  
print(c)
```

```
[[1 2]  
 [3 4]  
 [5 6]]  
[[1 3 5]  
 [2 4 6]]  
[[1 3 5]  
 [2 4 6]]
```

$$\begin{bmatrix} 3 & 4 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 2 \\ 4 & 1 \end{bmatrix}$$

陣列形狀與內容操作

- 新增陣列的維度
 - **NumPy**陣列的索引可以使用**np.newaxis**物件來新增陣列的維度，如下所示：

```
In [4]: a = np.array([1,2,3])  
        print(a)  
        b = a[:, np.newaxis]  
        print(b)  
        print(b.shape)
```

```
[1 2 3]  
[[1]  
 [2]  
 [3]]  
(3, 1)
```

陣列形狀與內容操作

- 陣列複製、填滿值和連接陣列

- **NumPy**陣列可以使用**copy()**方法複製出**1**個內容完全相同的全新陣列**b**，如下所示：

```
In [6]: b = a.copy()
        print(b)

[1 2 3]
```

- **fill()**方法可以指定陣列元素成為單一值，可以將**4**個陣列元素值都填成**4**，如下所示：

```
In [7]: b.fill(4)
        print(b)

[4 4 4]
```

- 如果有多個**NumPy**陣列，可以使用**np.concatenate()**方法來連接多個陣列

```
In [8]: c = np.concatenate((a,b))
        print(c)

[1 2 3 4 4 4]
```

陣列形狀與內容操作

- 連接多個二維陣列

- 在使用`np.concatenate()`方法連接多個二維陣列時，可以指定參數`axis`軸的連接方向，參數值`0`是直向（預設值），可以連接在目前二維陣列的下方，如下所示：

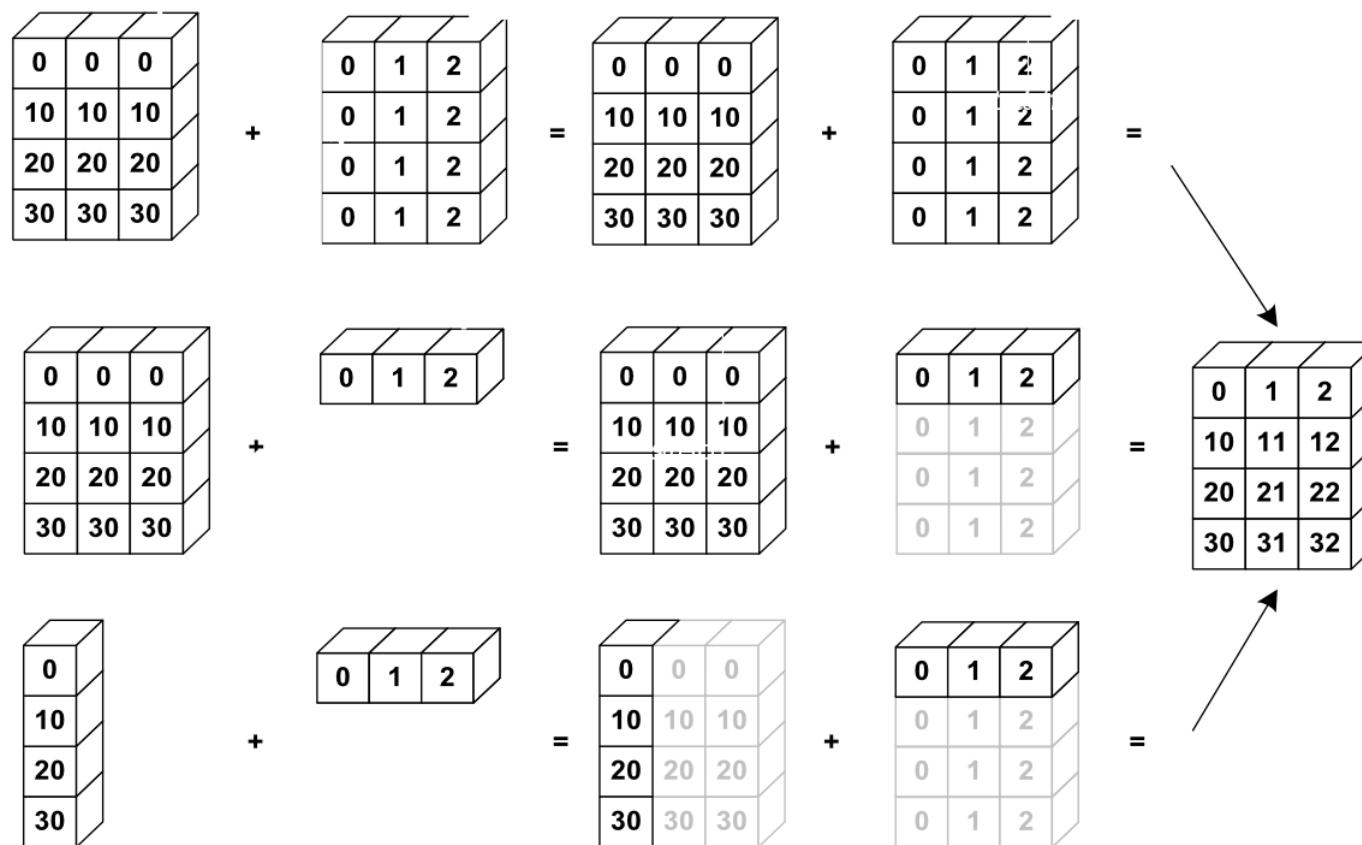
```
In [9]: a = np.array([[1,2],[3,4]])  
        b = np.array([[5,6],[7,8]])  
        c = np.concatenate((a,b), axis=0)  
        print(c)  
  
[[1 2]  
 [3 4]  
 [5 6]  
 [7 8]]
```

- `axis`軸的值`1`是橫向，每一個陣列是連接在目前陣列的右方，如下所示：

```
In [10]: c = np.concatenate((a,b), axis=1)  
         print(c)  
  
[[1 2 5 6]  
 [3 4 7 8]]
```

陣列廣播

- 「廣播」(**Broadcasting**) 可以讓不同形狀的陣列執行數學運算。



陣列廣播

- 使用廣播執行陣列相加運算

- **NumPy**廣播可以計算二維陣列和一維陣列的加法運算

建立二維陣列**a** (4 X 3) 和一維陣列**b** (尺寸3個元素) 後，進行**a + b**的加法運算

```
In [2]: a = np.array([[1,2,3],[4,5,6],[7,8,9],[10,11,12]])
print(a)
print("a 形狀: " + str(a.shape))
b = np.array([1,0,1])
print(b)
print("b 形狀: " + str(b.shape))
```

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
```

a 形狀: (4, 3)

```
[1 0 1]
```

b 形狀: (3,)

1	2	3
4	5	6
7	8	9
10	11	12

+

1	0	1
---	---	---

=

1	2	3
4	5	6
7	8	9
10	11	12

+

1	0	1
1	0	1
1	0	1
1	0	1

=

2	2	4
5	5	7
8	8	10
11	11	13

```
In [3]: c = a + b
print(c)
```

```
[[ 2  2  4]
 [ 5  5  7]
 [ 8  8 10]
 [11 11 13]]
```

NUMPY其他函數運算

```
>>> index = np.random.randint(0,len(x),5) #隨機整數值[low,high)
```

```
>>> index
```

```
array([9, 5, 9, 5, 6])
```

```
>>> noise = np.random.standard_normal(5)*0.3 #隨機雜訊
```

```
>>> noise
```

```
array([ 0.07000973, 0.17404817, 0.16620985, 0.15281169, -0.08693726])
```

```
>>> x[index]
```

```
array([ 90., 50., 90., 50., 60.])
```

```
>>> x[index] += noise #添加雜訊
```

```
>>> x[index]
```

```
array([ 90.16620985, 50.15281169, 90.16620985, 50.15281169, 59.91306274])
```

NUMPY其他函數運算

```
>>> x = np.arange(0, 100, 10, dtype=np.float64)
```

```
>>> np.sin(x)                #一維陣列中所有元素求正弦值  
array([ 0.          , -0.54402111,  0.91294525, -0.98803162,  0.74511316,  
       -0.26237485, -0.30481062,  0.77389068, -0.99388865,  0.89399666])
```

```
>>> b = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
>>> np.cos(b)                #二維陣列中所有元素求餘弦值  
array([[ 0.54030231, -0.41614684, -0.9899925 ],  
       [-0.65364362,  0.28366219,  0.96017029],  
       [ 0.75390225, -0.14550003, -0.91113026]])
```

```
>>> np.round(_)              #四捨五入  
array([[ 1., -0., -1.],  
       [-1.,  0.,  1.],  
       [ 1., -0., -1.]])
```

NUMPY其他函數運算

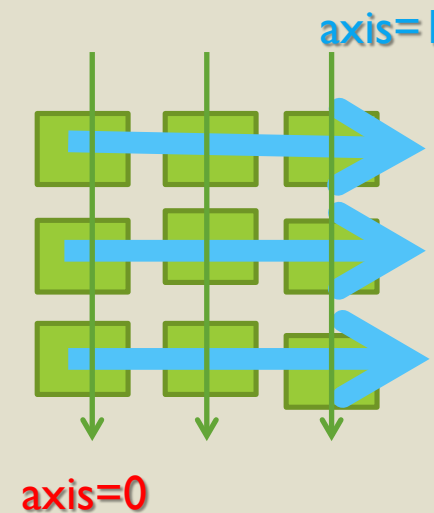
```
>>> x = np.random.rand(10)          # Random values in a given shape
>>> x                                # from a uniform distribution over [0, 1).
array([0.82541802, 0.04283783, 0.11830544, 0.98540916, 0.58533409,
       0.56195097, 0.97442386, 0.21045253, 0.62461573, 0.75924821])
>>> x = x*10

X = np.array([18.25418018, 10.42837828, 1.18305441, 9.85409158, 5.85334093,
              25.61950967, 49.74423864, 72.10452527, 6.24615732, 7.5924821 ])
>>> np.floor(x)                      #所有元素向下取最大整數(無條件捨去小數點)
array([18., 10., 1., 9., 5., 25., 49., 72., 6., 7.])

>>> np.ceil(x)                      #所有元素向上取最小整數(無條件進位到整數)
array([19., 11., 2., 10., 6., 26., 50., 73., 7., 8.])
```

指定不同維度上的計算

```
>>> x = np.arange(0,10).reshape(2,5)    #創建二維陣列
>>> x
array([[0, 1, 2, 3, 4],
       [5, 6, 7, 8, 9]])
>>> np.sum(x)                             #二維陣列所有元素求和
45
>>> np.sum(x, axis=0)                     #二維陣列縱向求和
array([ 5, 7, 9, 11, 13])
>>> np.sum(x, axis=1)                     #二維陣列橫向求和
array([10, 35])
>>> np.mean(x, axis=0)                    #二維陣列縱向計算算術平均值
array([ 2.5, 3.5, 4.5, 5.5, 6.5])
>>> weight = [0.3, 0.7]                  #設定權重
>>> np.average(x, axis=0, weights=weight) #二維陣列縱向計算加權平均值
array([ 3.5, 4.5, 5.5, 6.5, 7.5])
```



指定不同維度上的計算

```
>>> np.max(x) #所有元素最大值
```

```
9
```

```
>>> np.max(x, axis=0) #每列元素的最大值
```

```
array([5, 6, 7, 8, 9])
```

```
>>> x = np.random.randint(0, 10, size=(3,3)) #創建二維陣列
```

```
>>> x
```

```
array([[4, 9, 1],  
       [7, 4, 9],  
       [8, 9, 1]])
```

```
>>> np.std(x) # 對所有元素算一個標準差
```

```
3.1544599036840864
```

```
>>> np.std(x, axis=1) #每行元素的標準差
```

```
array([3.29983165, 2.05480467, 3.55902608])
```

指定不同維度上的計算

```
>>> np.var(x, axis=0) #每列元素的變異數
```

```
array([2.88888889, 5.55555556, 14.22222222])
```

```
>>> np.sort(x, axis=0) #縱向排序
```

```
array([[4, 4, 1],
```

```
       [7, 9, 1],
```

```
       [8, 9, 9]])
```

```
>>> np.sort(x, axis=1) #橫向排序
```

```
array([[1, 4, 9],
```

```
       [4, 7, 9],
```

```
       [1, 8, 9]])
```

RESHAPE 改變陣列大小

```
>>> a = np.arange(1, 11, 1)
```

```
>>> a
```

```
array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
```

```
>>> a.shape = 2, 5          #改為2行5列
```

```
>>> a
```

```
array([[ 1,  2,  3,  4,  5],  
       [ 6,  7,  8,  9, 10]])
```

```
>>> a.shape = 5, -1        #-1表示自動計算
```

```
>>> a
```

```
array([[ 1,  2],  
       [ 3,  4],  
       [ 5,  6],  
       [ 7,  8],  
       [ 9, 10]])
```

```
>>> b = a.reshape(2,-1)     #reshape()方法返回新陣列
```

```
>>> b
```

```
array([[ 1,  2,  3,  4,  5],  
       [ 6,  7,  8,  9, 10]])
```

只取小數

```
>>> x = np.random.rand(10)*50
```

```
>>> x
```

```
array([ 0.69708323, 14.99931488, 15.04431214, 24.60547929,  
       12.12020273, 42.72638176, 16.01128916, 38.91558471,  
       39.6877989 , 21.98678429])
```

```
>>> np.array([t-int(t) for t in x])
```

```
array([ 0.69708323, 0.99931488, 0.04431214, 0.60547929, 0.12020273,  
       0.72638176, 0.01128916, 0.91558471, 0.6877989 , 0.98678429])
```

NUMPY基礎教學

```
>>> x
array([0, 4, 3, 3, 8, 4, 7, 3, 1, 7])
>>> y
array([0, 4, 3, 5, 6, 4, 1, 2, 1, 7])
>>> np.argmax(x)    #最大值的索引
4
>>> np.where(x!=y)[0]  #x!=y的地方
array([3, 4, 6, 7])

>>> np.where(x<5, 0, 1)  #小於5的元素值對應0，其他對應1
array([[0, 0, 0, 0, 1, 0, 1, 0, 0, 1]])

#小於4的元素乘以2，大於7的元素乘以3，其他元素變為0
>>> np.piecewise(x,[x<4, x>7],[lambda x:x*2,lambda x:x*3])
array([[ 0,  0,  6,  6, 24,  0,  0,  6,  2,  0]])
```

NUMPY基礎教學

- 計算唯一值以及出現次數

```
>>> x = np.random.randint(0,10,7)
```

```
>>> x
```

```
array([8, 7, 7, 5, 3, 8, 0])
```

```
>>> np.bincount(x) #元素出現次數，0出現1次
```

```
array([1, 0, 0, 1, 0, 1, 0, 2, 2], dtype=int64) #1、2沒出現，3出現1次，以此類推
```

```
>>> np.sum(_) #所有元素出現次數之和等於陣列長度
```

```
>>> len(x)
```

```
>>> np.unique(x) #返回唯一元素值
```

```
array([0, 3, 5, 7, 8])
```

```
>>> x = numpy.random.randint(0,10,[2,7])
```

```
>>> x
```

```
array([[4, 0, 3, 3, 7, 8, 5],  
       [7, 3, 0, 9, 5, 1, 6]])
```

```
>>> x.flatten() #高維度矩陣平鋪
```

A decorative wavy line in a light green color runs vertically along the left side of the slide, separating a light beige background from the dark teal background.

PYTHON PACKAGE - PANDAS

PART 2

認識PANDAS

- Pandas套件簡介

- 名稱是源於"**Python and data analysis**" and "**panel data**"字首的縮寫，**Pandas**完整包含**NumPy**、**Scipy**和**Matplotlib**套件的功能

- 目的是幫助開發者能夠有效率的對資料進行篩選及分析，提供了快速、靈活、明確的資料結構，旨在簡單、直觀地處理關係型、標記型資料。

- **Pandas**套件是資料處理和分析工具，可以將**Pandas**套件視為是一套**Python**程式版的**Excel**試算表工具，透過**Python**程式碼針對表格資料執行試算表的功能。

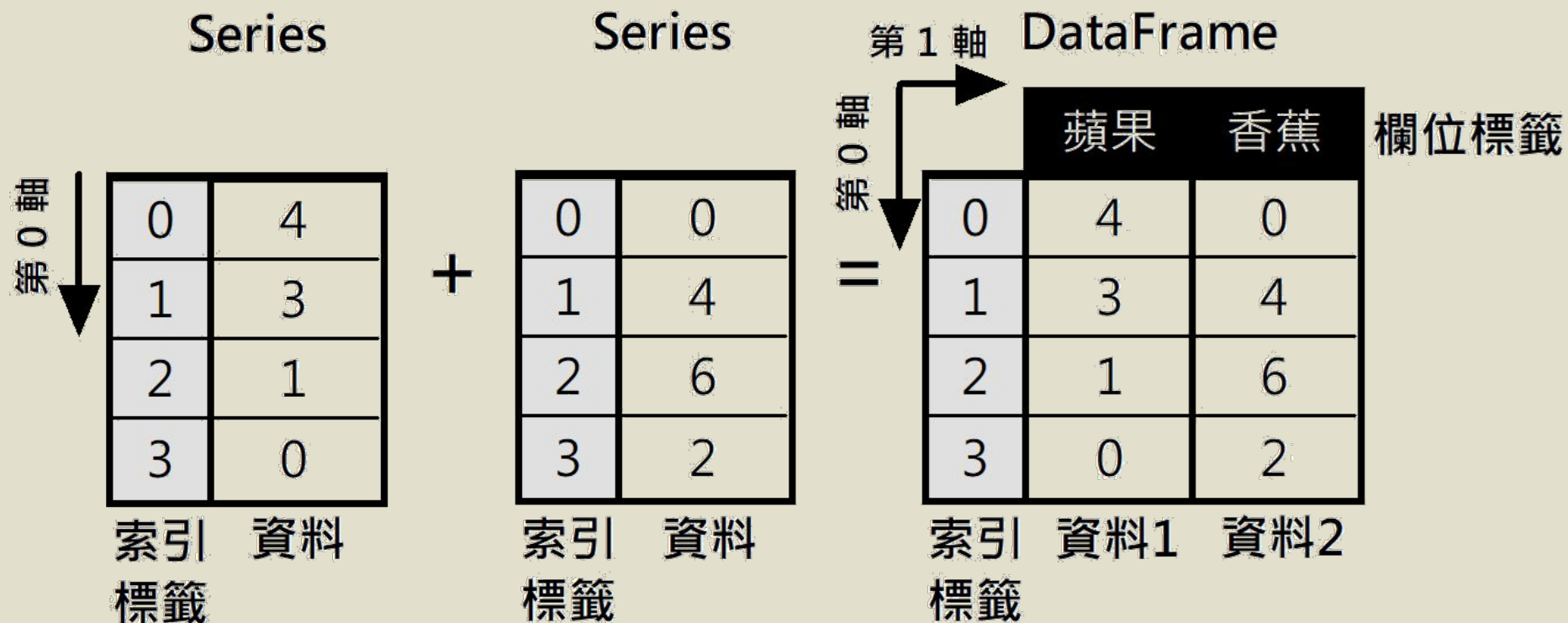
- 能夠快速地進行對資料的前處理，例如缺失值、異質，進行取代或刪除

- 能夠從檔案中讀取資料，在整理好後，也能進行輸出

Series			Series			DataFrame	
peppers			carrots			peppers	carrots
0	3	+	0	0	=	0	3
1	2		1	3		1	2
2	0		2	7		2	0
3	1		3	2		3	1

PANDAS的資料結構

- Pandas主要有兩種資料結構**Series**和**DataFrame**，如下圖所示：



SERIES物件

- 因為**Pandas**套件關於資料處理和分析的重點是**DataFrame**物件，所以本章只準備簡單說明**Series**物件的使用
- 在**Python**程式首先需要匯入**Pandas**套件的別名**pd**，如下所示：

```
In [1]: import pandas as pd
```

- 建立**Series**物件
 - **Series**物件可以使用**Python**清單來建立，使用的是**Series()**方法，參數是**Python**清單，如下所示：

```
In [2]: s = pd.Series([12, 29, 72, 4])  
print(s)
```

```
0    12  
1    29  
2    72  
3     4  
dtype: int64
```

SERIES物件

- 建立自訂索引的**Series**物件
 - **Series**物件如同是**2**個陣列，一是索引標籤；一是資料，所以可以使用**2**個**Python**清單建立**Series**物件，在**Series()**方法的第**1**個參數是資料清單，第**2**個是使用**index**參數指定的索引清單
 - 然後使用**index**屬性顯示索引；**values**屬性可以顯示資料

```
In [3]: fruits = ["蘋果", "橘子", "梨子"]
        quantities = [15, 33, 45]
        s = pd.Series(quantities, index=fruits)
        print(s)
```

```
蘋果    15
橘子    33
梨子    45
dtype: int64
```

```
In [4]: print(s.index)
        print(s.values)
```

```
Index(['蘋果', '橘子', '梨子'], dtype='object')
[15 33 45]
```

SERIES物件

- 使用索引取出資料和執行運算

- 在建立Series物件後，可以使用索引值來取出資料
- 如同NumPy陣列，一樣可以使用索引清單一次就取出多筆資料
- Series物件也可以作為運算元來執行四則運算，如下所示：

```
In [4]: print(s.index)
        print(s.values)

Index(['蘋果', '橘子', '梨子'], dtype='object')
[15 33 45]
```

```
In [5]: print(s["橘子"])
```

33

```
In [6]: print(s[["橘子", "梨子"]])
```

橘子 33
梨子 45
dtype: int64

```
In [6]: print((s+2)*3)
```

蘋果 51
橘子 105
梨子 141
dtype: int64

建立DATAFRAME物件

- 使用Python字典建立DataFrame物件
 - DataFrame物件是表格擁有列和欄標籤，首先建立3個元素的fruits字典，鍵是字串；值是清單，然後呼叫pd.DataFrame()方法建立DataFrame物件，如下所示：

```
In [2]: fruits = {"蘋果": [4, 3, 1, 0],  
                  "香蕉": [0, 4, 6, 2],  
                  "橘子": [1, 5, 2, 4]}  
df = pd.DataFrame(fruits)  
df
```

Out[2]:

	蘋果	香蕉	橘子
0	4	0	1
1	3	4	5
2	1	6	2
3	0	2	4

建立DATAFRAME物件

- 使用自訂索引建立DataFrame物件

- 如果沒有指明索引標籤，**Pandas**預設替**DataFrame**物件產生數值索引標籤，當然可以使用**cites**清單來建立自訂索引標籤，共有**4**個元素，對應**4**筆資料。
- 在建立**DataFrame**物件時，可以使用**columns**參數**重新指定欄位順序**，將原來蘋果、香蕉、橘子順序改為香蕉、橘子、蘋果。

```
In [3]: cites = ["台北", "新北", "台中", "高雄"]  
df = pd.DataFrame(fruits, index=cites)  
df
```

Out[3]:

	蘋果	香蕉	橘子
台北	4	0	1
新北	3	4	5
台中	1	6	2
高雄	0	2	4

```
In [4]: df = pd.DataFrame(fruits,  
                           columns=["香蕉", "橘子", "蘋果"],  
                           index=cites)  
df
```

Out[4]:

	香蕉	橘子	蘋果
台北	0	1	4
新北	4	5	3
台中	6	2	1
高雄	2	4	0

建立DATAFRAME物件

- 更改索引標籤和欄位標籤

- 在建立**DataFrame**物件**df**後，可以使用**columns**屬性重新指定欄位標籤，**index**屬性是更改索引標籤，如下所示：

```
In [5]: df.columns = ["banana", "orange", "apple"]  
cites[2] = "桃園"  
df.index = cites  
df
```

Out[5]:

	banana	orange	apple
台北	0	1	4
新北	4	5	3
桃園	6	2	1
高雄	2	4	0

建立DATAFRAME物件

- 轉置DataFrame物件

- 可以使用.T屬性轉置DataFrame物件，即欄變列、列成欄

```
In [6]: df.T
```

```
Out[6]:
```

	台北	新北	桃園	高雄
banana	0	4	6	2
orange	1	5	2	4
apple	4	3	1	0

匯入與匯出 **DATAFRAME** 物件

- **Pandas** 套件可以匯入和匯出多種格式檔案至 **DataFrame** 物件，匯出 **DataFrame** 物件 **df** 至檔案的相關方法，如下表所示：

方法	說明
df.to_csv(filename)	匯出成 CSV 格式的檔案
df.to_json(filename)	匯出成 JSON 格式的檔案
df.to_html(filename)	匯出成 HTML 表格標籤的檔案
df.to_excel(filename)	匯出成 Excel 檔案

匯入與匯出DATAFRAME物件

- 匯入檔案內容成為**DataFrame**物件**df**的相關方法，如下表所示：

方法	說明
<code>df.read_csv(filename)</code>	匯入 CSV 格式的檔案
<code>df.read_json(filename)</code>	匯入 JSON 格式的檔案
<code>df.read_html(filename)</code>	匯入 HTML 檔案， Pandas 會抽出<table>表格標籤的資料
<code>df.read_excel(filename)</code>	匯入 Excel 檔案

顯示DATAFRAME資訊與取出資料

- 當建立或匯入檔案成為**DataFrame**物件後，可以馬上使用相關方法和屬性來顯示**DataFrame**物件的基本資訊和取出資料
 - **DataFrame**物件的每一列是一筆記錄，每一欄是記錄的欄位，可以使用**head()**方法顯示**前幾筆記錄**，預設是**5筆**，如下所示：
 - **DataFrame**物件可以使用**tail()**方法顯示最後幾筆記錄，預設也是**5筆**，第**2**個指定參數**2**，可以顯示**最後2筆**記錄，如下所示：

```
In [2]: df.head()
```

```
Out[2]:
```

	蘋果	香蕉	橘子
0	4	0	1
1	3	4	5
2	1	6	2
3	0	2	4

```
In [4]: print(df.tail())  
print(df.tail(2))
```

	蘋果	香蕉	橘子
0	4	0	1
1	3	4	5
2	1	6	2
3	0	2	4

	蘋果	香蕉	橘子
2	1	6	2
3	0	2	4

顯示DATAFRAME資訊與取出資料

- 取得**DataFrame**物件的索引、欄位和資料
 - **DataFrame**物件可以使用**index**、**columns**和**values**屬性取得索引、欄位標籤和資料（不含索引和欄位標籤），如下所示：

```
In [5]: print(df.index)
        print(df.columns)
        print(df.values)
```

```
RangeIndex(start=0, stop=4, step=1)
Index(['蘋果', '香蕉', '橘子'], dtype='object')
[[4 0 1]
 [3 4 5]
 [1 6 2]
 [0 2 4]]
```

顯示DATAFRAME資訊與取出資料

- 顯示DataFrame物件的摘要資訊
 - DataFrame物件可以使用Python的len()函數來取得記錄數，shape屬性取得形狀，如下所示：

```
In [6]: print(len(df))  
        print(df.shape)
```

4

(4, 3)

```
In [7]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 4 entries, 0 to 3  
Data columns (total 3 columns):  
#   Column  Non-Null Count  Dtype  
---  ---  
0   蘋果    4 non-null      int64  
1   香蕉    4 non-null      int64  
2   橘子    4 non-null      int64  
dtypes: int64(3)  
memory usage: 224.0 bytes
```

走訪DATAFRAME物件

- 因為**DataFrame**物件是類似表格的試算表物件，如同關聯式資料庫的資料表，每一列相當是一筆記錄，可以使用**for**迴圈走訪**DataFrame**物件的每一筆記錄。
- 在**DataFrame**物件可以使用**iterrows()**方法走訪每一筆記錄，如下所示：

```
In [2]: for index, row in df.iterrows():  
        print(index, row["蘋果"], row["香蕉"],  
              row["橘子"])
```

```
0 4 0 1  
1 3 4 5  
2 1 6 2  
3 0 2 4
```

選擇、過濾與排序資料

- 接下來範例都是使用相同**DataFrame**物件**df**，和指定自訂索引**ordinals**清單，如下所示：

In [1]: `import pandas as pd`

```
products = {"channel": ["網路", "網路", "電視", "電視", "郵購", "郵購"],  
            "company": ["EHS", "Momo", "EHS", "Viva", "Momo", "EHS"],  
            "sales":    [11.22, 23.50, 12.99, 15.95, 25.75, 11.55]}  
ordinals = ["A", "B", "C", "D", "E", "F"]  
df = pd.DataFrame(products, index=ordinals)  
df
```

Out[1]:

	channel	company	sales
A	網路	EHS	11.22
B	網路	Momo	23.50
C	電視	EHS	12.99
D	電視	Viva	15.95
E	郵購	Momo	25.75
F	郵購	EHS	11.55

選取資料

- 使用欄位標籤選取單一或多個欄位
 - **DataFrame**物件可以直接使用欄位標籤，或欄位標籤清單來選取單一欄位的**Series**物件或多欄位的**DataFrame**物件，如下所示：

```
In [2]: df["sales"].head(3)
```

```
Out[2]: A    11.22  
       B    23.50  
       C    12.99  
       Name: sales, dtype: float64
```

- 使用欄位標籤清單來同時選取多個欄位，如下所示：

```
In [4]: df[["channel", "company"]].head(3)
```

```
Out[4]:
```

	channel	company
A	網路	EHS
B	網路	Momo
C	電視	EHS

選取資料

- 使用索引標籤選取特定範圍的記錄
 - 對於**DataFrame**物件每一列的記錄來說，可以使用從**0**開始的索引標籤，或使用自訂索引標籤來選取特定範圍的記錄，首先是預設數字索引的範圍，可以選取第**1~2**筆記錄，但不含索引值**2**的第**3**筆
 - 如果使用自訂索引標籤，例如：索引**"C"**到**"E"**範圍，就會包含最後一筆，即**"E"**，

```
In [5]: df[0:2]
```

```
Out[5]:
```

	channel	company	sales
A	網路	EHS	11.22
B	網路	Momo	23.50

```
In [6]: df["C":"E"]
```

```
Out[6]:
```

	channel	company	sales
C	電視	EHS	12.99
D	電視	Viva	15.95
E	郵購	Momo	25.75

選取資料

- 使用**loc**索引器選取資料

- 在**DataFrame**二維表格定位資料時，**loc**索引器可以使用索引和欄位標籤來取出表格的部分資料，其語法如下所示：

df.loc[索引標籤]

df.loc[[索引標籤1, 索引標籤2,...]]

df.loc[索引標籤, 欄位標籤]

df.loc[[索引標籤1, 索引標籤2,...], [欄位標籤1, 欄位標籤2...]]

df.loc[索引標籤1:索引標籤2, 欄位標籤]

df.loc[索引標籤1:索引標籤2, [欄位標籤1, 欄位標籤2...]]

df.loc[索引標籤1:索引標籤2, 欄位標籤1:欄位標籤2]

- 使用**iloc**索引器選取資料

- **DataFrame**物件的**iloc**索引器是使用從**0**開始的索引位置值來選取資料，其語法如下所示：

df.iloc[列索引位置, 欄索引位置]

選取資料

- 使用**loc**索引器選取資料
 - 首先使用**loc**索引器以索引標籤選取指定記錄

```
In [7]: df.loc["B"]  
Out[7]: channel    網路  
        company    Momo  
        sales      23.5  
        Name: B, dtype: object
```

- 選取**"channel"**和**"sales"**欄位標籤的所有記錄

```
In [8]: df.loc[:, ["channel", "sales"]]  
Out[8]:
```

	channel	sales
A	網路	11.22
B	網路	23.50
C	電視	12.99
D	電視	15.95
E	郵購	25.75
F	郵購	11.55

選取資料

- 使用*iloc*索引器選取資料
 - **df.iloc**[列索引位置, 欄索引位置]
 - **iloc**索引位置除單一值外，也可以是「:」範圍，其操作方式就是Python切割運算子。第1個範例是索引值3的第4筆記錄，如下所示：

```
In [13]: df.iloc[3]
Out[13]: channel      電視
         company      Viva
         sales      15.95
         Name: D, dtype: object
```

過濾資料

- 使用布林條件和`isin()`方法來過濾資料
 - 在**DataFrame**物件的欄位標籤可以使用布林條件，只選擇條件成立的記錄資料。
- 使用多個條件來過濾資料
 - 在布林索引可以使用多個條件，例如：**sales**大於**15**，且小於**25**，這是使用「&」的**And**邏輯運算子，如下所示：

```
In [2]: df[df.sales > 15]
```

```
Out[2]:
```

	channel	company	sales
B	網路	Momo	23.50
D	電視	Viva	15.95
E	郵購	Momo	25.75

```
In [4]: df[(df.sales > 15) & (df.sales < 25)]
```

```
Out[4]:
```

	channel	company	sales
B	網路	Momo	23.50
D	電視	Viva	15.95

排序資料

- 指定欄位標籤來排序記錄資料
 - **DataFrame**物件可以呼叫**sort_values()**方法，指定特定欄位標籤來排序記錄資料。

```
In [2]: df.sort_values("sales", ascending=False)
```

```
Out[2]:
```

	channel	company	sales
E	郵購	Momo	25.75
B	網路	Momo	23.50
D	電視	Viva	15.95
C	電視	EHS	12.99
F	郵購	EHS	11.55
A	網路	EHS	11.22

排序資料

- 排序索引或欄位標籤
 - **DataFrame**物件使用**set_index()**方法可以指定索引標籤是哪一個欄位標籤後，即可呼叫**sort_index()**方法排序目前的索引標籤，看到索引標籤從大至小排序，因為**axis**屬性值是**0**。

```
In [4]: df.set_index("sales", inplace=True)  
df.sort_index(axis=0, ascending=False)
```

Out[4]:

	channel	company
sales		
25.75	郵購	Momo
23.50	網路	Momo
15.95	電視	Viva
12.99	電視	EHS
11.55	郵購	EHS
11.22	網路	EHS

更新資料

- 更新純量值
 - 當使用**loc**或**iloc**索引器選取資料後，就可以更新選取的資料。
- 更新單筆記錄
 - 在使用**Python**清單建立新記錄後，可以選取欲取代的記錄來取代成新記錄。

```
In [2]: df.loc["A", "sales"] = 9.6  
df.iloc[1,2] = 22.01  
df.head(2)
```

Out[2]:

	channel	company	sales
A	網路	EHS	9.60
B	網路	Momo	22.01

```
In [3]: r = ["郵購", "Viva", 18.9]  
df.loc["C"] = r  
df.head(3)
```

Out[3]:

	channel	company	sales
A	網路	EHS	9.60
B	網路	Momo	22.01
C	郵購	Viva	18.90

更新資料

- 更新整個欄位
 - 同樣可以建立**Python**清單來更新整個欄位，

```
In [4]: s = [10, 20, 30, 40, 50, 60]
df.loc[ : , "sales"] = s
df.head()
```

Out[4]:

	channel	company	sales
A	網路	EHS	10
B	網路	Momo	20
C	郵購	Viva	30
D	電視	Viva	40
E	郵購	Momo	50

刪除資料

- 刪除純量值
 - 如同更新純量值，刪除只是將指定成**None**（或NumPy套件的**np.nan**）。
- 刪除單筆和多筆記錄
 - **DataFrame**物件可以使用**drop()**方法來刪除記錄，參數是索引標籤或數字索引，如果是索引標籤清單，就是刪除多筆。首先第**2**筆和第**4**筆的**B**和**D**。

```
In [2]: df.loc["A", "sales"] = None  
df.iloc[1,2] = None  
df.head(3)
```

Out[2]:

	channel	company	sales
A	網路	EHS	NaN
B	網路	Momo	NaN
C	電視	EHS	12.99

```
In [3]: df2 = df.drop(["B", "D"]) # 2,4 筆  
df2.head()
```

Out[3]:

	channel	company	sales
A	網路	EHS	NaN
C	電視	EHS	12.99
E	郵購	Momo	25.75
F	郵購	EHS	11.55

刪除資料

- 刪除整個欄位

- 刪除欄位也是使用**drop()**方法，只是需要指定**axis**參數值是**1**，例如：刪除整個**sales**欄位。

```
In [5]: df2 = df.drop(["sales"], axis=1)
df2.head(3)
```

Out[5]:

	channel	company
A	網路	EHS
B	網路	Momo
C	電視	EHS

新增資料

- 新增一筆記錄
 - **DataFrame**物件新增記錄（列）只需指定一個不存在的索引標籤，就可以新增記錄。
- 新增整個欄位
 - 在**DataFrame**物件只需指定一個不存在的欄位標籤，就可以新增欄位，我們可以使用**Series**物件或**NumPy**陣列來指定欄位值。

```
In [2]: df.loc["H"] = ["網路", "Viva", 16.5]  
df.tail(3)
```

Out[2]:

	channel	company	sales
E	郵購	Momo	25.75
F	郵購	EHS	11.55
H	網路	Viva	16.50

```
In [5]: from numpy.random import randint
```

```
df.loc[:, "items"] = randint(100, 120, size=len(df))  
df.head()
```

Out[5]:

	channel	company	sales	quantity	items
A	網路	EHS	11.22	6390	101
B	網路	Momo	23.50	8289	117
C	電視	EHS	12.99	8691	100
D	電視	Viva	15.95	7247	100
E	郵購	Momo	25.75	6078	111

群組、樞紐分析與統計函數

- 「群組」(**Grouping**) 是先將資料依條件分類成群組後，再套用相關方法在各群組來取得統計資料。欲使用群組來計算總計，首先使用字典建立測試的**DataFrame**物件，如下所示：

	名稱	編號	數量	售價
0	客戶A	訂單1	4	495
1	客戶B	訂單1	4	496
2	客戶A	訂單2	1	360
3	客戶B	訂單3	2	451
4	客戶A	訂單2	3	221
5	客戶B	訂單2	4	321
6	客戶A	訂單1	2	466
7	客戶A	訂單3	1	260

```
data = {  
    "名稱":["客戶A","客戶B","客戶A","客戶B","客  
    戶A","客戶B","客戶A","客戶A"],  
    "編號":["訂單1","訂單1","訂單2","訂單3","訂  
    單2","訂單2","訂單1","訂單3"],  
    "數量":[4,4,1,2,3,4,2,1],  
    "售價":[495,496,360,451,221,321,466,260]}
```

群組

- 群組

- 名稱和編號欄位都有重複資料，我們可以分別使用這些欄位來群組記錄後，呼叫 `sum()` 方法計算欄位值的總和。

```
In [2]: df.groupby("名稱").sum()
```

```
Out[2]:
```

	數量	售價
名稱		
客戶A	11	1802
客戶B	10	1268

樞紐分析表

- 樞紐分析表

- **DataFrame**物件是呼叫**pivot_table()**方法產生樞紐分析表，資料為左下表格新增**score**欄位。

	channel	company	sales
A	網路	EHS	11.22
B	網路	Momo	23.50
C	電視	EHS	12.99
D	電視	Viva	15.95
E	郵購	Momo	25.75
F	郵購	EHS	11.55

```
In [2]: score = [4, 3, 5, 7, 5, 8]
df["score"] = score
df
```

Out[2]:

	channel	company	sales	score
A	網路	EHS	11.22	4
B	網路	Momo	23.50	3
C	電視	EHS	12.99	5
D	電視	Viva	15.95	7
E	郵購	Momo	25.75	5
F	郵購	EHS	11.55	8

樞紐分析表

- 使用**pivot_table()**方法以欄位值為標籤來重塑**DataFrame**物件的形狀，也就是建立樞紐分析表

```
In [3]: pivot_tb = df.pivot_table(index='channel',  
                                   columns='company',  
                                   values='sales')  
  
pivot_tb
```

Out[3]:

company	EHS	Momo	Viva
channel			
網路	11.22	23.50	NaN
郵購	11.55	25.75	NaN
電視	12.99	NaN	15.95

統計的相關方法

- **describe()**方法

- 顯示**DataFrame**物件指定欄位，或**Series**物件的資料描述。

```
In [2]: df["sales"].describe()
```

```
Out[2]: count      6.000000
        mean      16.826667
        std       6.307547
        min      11.220000
        25%      11.910000
        50%      14.470000
        75%      21.612500
        max      25.750000
        Name: sales, dtype: float64
```

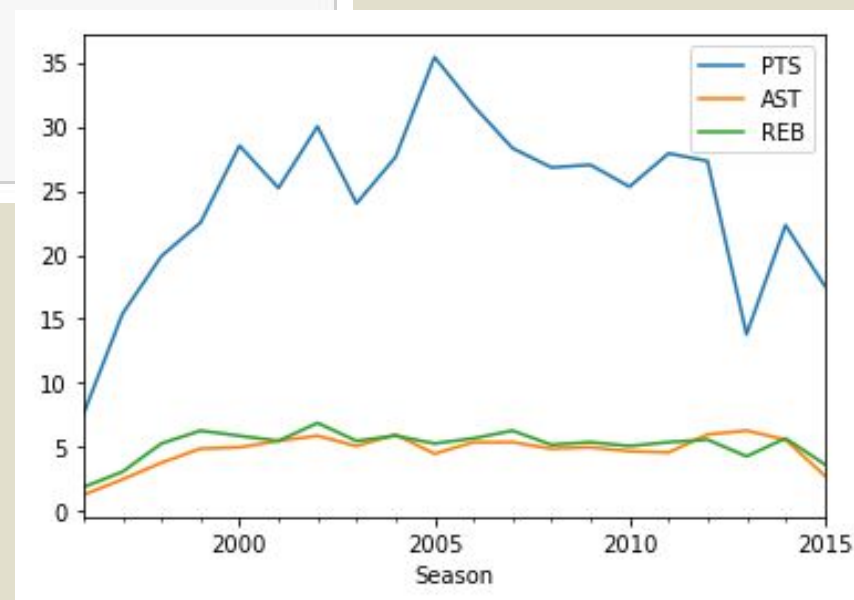
方法	說明
count()	非NaN值計數
mode()	眾數
median()	中位數
quantile()	四分位數，分別是quantile(q=0.25)、quantile(q=0.5)、quantile(q=0.75)
mean()	平均數
max()	最大值
min()	最小值
sum()	總和
var()	變異數
std()	標準差
cov()	共變異數
corr()	相關係數
cumsum()	累積總和
cumprod()	累積乘積

PANDAS資料視覺化

- 建立**DataFrame**或**Series**物件後，就可以使用**plot()**方法繪製長條圖，

```
In [2]: df = pd.read_csv("Kobe_stats.csv")
data = pd.DataFrame()
data["Season"] = pd.to_datetime(df["Season"])
data["PTS"] = df["PTS"]
data["AST"] = df["AST"]
data["REB"] = df["TRB"]
data = data.set_index("Season")

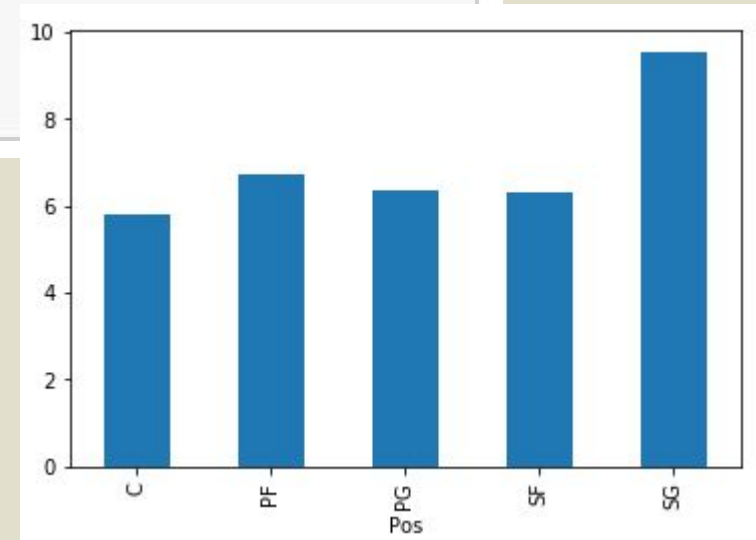
data.plot(kind="line")
```



PANDAS資料視覺化

- **NBA**火箭隊各位置平均得分的長條圖，在讀取**CSV**檔案後，計算各位置的得分平均，這是**Series**物件**points**，然後使用**Series**物件建立**DataFrame**物件**data**，

```
In [3]: df = pd.read_csv("HOU_players_stats.csv")
df_grouped = df.groupby("Pos")
points = df_grouped["PTS/G"].mean()
data = pd.DataFrame()
data["Points"] = points
points.plot(kind="bar")
```



PANDAS資料視覺化

with Numpy

建立一筆隨機的資料`np.random.randn(100, 4)`，`x`為日期，`columns`設定為A,B,C,D將資料累加

```
In [1]: import pandas as pd
import numpy as np

df = pd.DataFrame(np.random.randn(100, 4), index=pd.date_range('05/20/2021', periods=100), columns=list('ABCD'))
```

```
In [2]: df = df.cumsum()
```

```
In [3]: df.plot()
```

```
Out[3]: <AxesSubplot:>
```

